

CSCI 232 Assignment 04 – Linked List in C

Points: 10

Instructions

1. **Fork** the repository on GitHub: <https://github.com/jpach-cs/232-ass04-template>
2. **Clone** your forked repository to your local machine.
3. Open `src/code.c` and `src/tests.c`.
4. Uncomment the author block at the top of `src/code.c` and fill in your name:

```
// char *AUTHOR_NAME      = "Your Name";  
// char *AUTHOR_AUTHORSHP = "I acknowledge that I have worked on this  
// assignment independently...";
```

5. Work through each function in **two parts**:

Part 1 — `src/code.c`

- Read the description and hint for each function.
- Implement the function body.
- Do **not** modify function signatures.
- Private functions are marked **static** — implement them first, they are called internally by public functions.

Part 2 — `src/tests.c`

- Read the description for each test function.
- Implement the test body using `TEST_ASSERT_TRUE_MESSAGE`.
- Stack tests: declare `Node` variables directly — no `malloc`.
- Heap tests: use `createNode` and `destroyList` for cleanup.
- Do **not** modify function names or signatures.

6. Build and run locally:

`make`

`bin/main.exe`

7. When all local tests pass — **commit** and **push** to your repository.
 8. Go to the **Actions** tab on GitHub and verify the CI results.
-

Private functions — `src/code.c`

These are **static** — internal helpers, not visible outside `code.c`. Implement them first — public functions depend on them.

Function	Description
<code>_nullify</code>	Set pointer to NULL via double pointer
<code>_findFirst</code>	Return the first node (head)
<code>_findLast</code>	Walk to the last node and return it
<code>_findValue</code>	Walk and return first node matching value

Public functions — `src/code.c`

Function	Description
<code>initNode</code>	Initialize an existing node — set value, clear nextPtr
<code>createNode</code>	Allocate a new node on the heap via <code>malloc</code>
<code>destroyNode</code>	Free a heap node and set pointer to NULL
<code>destroyList</code>	Free all nodes in the list one by one
<code>addFirst</code>	Insert node at the front of the list
<code>addLast</code>	Insert node at the end of the list
<code>detachFirst</code>	Remove and return the first node — no free
<code>detachLast</code>	Remove and return the last node — no free
<code>detachValue</code>	Find, remove and return a node by value — no free
<code>deleteFirst</code>	Remove and free the first node
<code>deleteLast</code>	Remove and free the last node
<code>deleteValue</code>	Find, remove and free a node by value
<code>printList</code>	Print all values one per line
<code>listLength</code>	Return the number of nodes in the list

`detach` = remove from list, caller decides memory. `delete` = remove from list AND free — heap only. `delete` is always: `detach` + `destroyNode`.

Tests to implement — `src/tests.c`

Stack tests — use Node variables on the stack

Test function	What to verify
<code>test_initNode_sets_value</code>	value field contains the given value
<code>test_initNode_sets_next_null</code>	nextPtr is NULL after call
<code>test_initNode_null_guard</code>	NULL pointer does not crash
<code>test_detachFirst_returns_node</code>	returned pointer equals head node
<code>test_detachFirst_updates_head</code>	headPtr moves to second node
<code>test_detachFirst_empty_list</code>	returns NULL for empty list

Test function	What to verify
<code>test_detachValue_found</code>	correct node returned, chain reconnected
<code>test_detachValue_head</code>	head node correctly detached
<code>test_detachValue_not_found</code>	returns NULL when value missing

Heap tests — use `createNode` and `destroyList`

Test function	What to verify
<code>test_createNode_not_null</code>	returned pointer is not NULL
<code>test_createNode_value</code>	value field contains the given value
<code>test_createNode_next_null</code>	<code>nextPtr</code> is NULL after call
<code>test_destroyNode_sets_null</code>	pointer is NULL after <code>destroyNode</code>
<code>test_addFirst_empty_list</code>	<code>headPtr</code> points to new node
<code>test_addFirst_non_empty</code>	new node is at the front
<code>test_addFirst_null_headptr</code>	returns -1 when <code>headPtrPtr</code> is NULL
<code>test_addLast_empty_list</code>	<code>headPtr</code> points to new node
<code>test_addLast_non_empty</code>	new node is at the end
<code>test_addLast_null_guard</code>	returns -1 when <code>headPtrPtr</code> is NULL
<code>test_deleteFirst_removes_node</code>	returns 0, <code>headPtr</code> updated
<code>test_deleteFirst_empty_list</code>	returns -1 for empty list
<code>test_deleteValue_found</code>	returns 0, list length decreases
<code>test_deleteValue_not_found</code>	returns -1, list unchanged
<code>test_destroyList_empty_list</code>	<code>headPtr</code> is NULL after call
<code>test_listLength_empty</code>	returns 0 for NULL
<code>test_listLength_three</code>	returns 3 for three-node list
<code>test_printList_empty</code>	returns -1 for empty list

Rules

- Do **not** modify function signatures in `code.c` or `tests.c`
- Do **not** call private `_functions` from `tests.c`
- Stack tests: no `malloc`, no `free`
- Heap tests: always clean up with `destroyList` or `destroyNode`
- CI runs your `tests.c` together with instructor tests — both must pass

Grading

Component	Points
All functions correctly implemented	5
All test functions correctly written	5
Total	10

Author block — required

```
char *AUTHOR_NAME      = "Your Name";
char *AUTHOR_AUTHORSIP = "I acknowledge that I have worked on this
assignment independently, except where explicitly noted and referenced.
Any collaboration or use of external resources has been properly cited.
I am fully aware of the consequences of academic dishonesty and agree
to abide by the university's academic integrity policy.";
```